



SPRING BOOT CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & STARTER

Installation

```
<parent>spring-boot-starter-parent</parent>
<dep>spring-boot-starter-web</dep>
```

Bootstrap via start.spring.io

BOM manages all dependency versions

```
mvn spring-boot:run / ./gradlew bootRun
```

04. SPRING DATA JPA

Workflow

```
@Entity public class User { @Id @GeneratedValue...
interface UserRepo extends JpaRepository<User...
List<User> findByEmail(String email);
```

Derives queries from method names

```
@Query("SELECT u FROM User u WHERE u.active=t...
```

02. MAIN CLASS

Architecture

```
@SpringBootApplication
public class App {
    public static void main(String[] a) {
        SpringApplication.run(App.class, a);
    }
}
```

Combines @Config+@EnableAutoConfig+@Scan

05. DEPENDENCY INJECTION

CRUD API

```
@Service / @Repository / @Component
@Autowired // field injection (avoid)
```

Constructor injection preferred:

```
public UserSvc(UserRepo repo){ this.repo=repo...
@Bean // manual bean in @Configuration class
```

03. REST CONTROLLER

YAML / Config

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/{id}")
    public User get(@PathVariable Long id){...}
    @PostMapping
    public User create(@RequestBody User u){...}
```

06. CONFIGURATION

Integration

```
application.properties / application.yml
server.port=8080
spring.datasource.url=jdbc:postgresql://...
@Value("${app.secret}") private String secre...
@ConfigurationProperties(prefix="app")
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. PROFILES

Hardening

```
application-dev.yml / application-prod.yml
spring.profiles.active=dev
@Profile("prod") class ProdBean {}
-Dspring.profiles.active=prod (JVM arg)
SPRING_PROFILES_ACTIVE=prod (env var)
```

10. ACTUATOR

Unit Tests

```
<dep>spring-boot-starter-actuator</dep>
management.endpoints.web.exposure.include=*
GET /actuator/health /actuator/metrics
GET /actuator/env /actuator/beans
Custom: implement HealthIndicator interface
```

08. EXCEPTION HANDLING

Observability

```
@RestControllerAdvice
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<Error> handle(ex) {
    return ResponseEntity.status(404)...
}
@ResponseStatus(HttpStatus.NOT_FOUND)
```

11. LOGGING

Commands

```
private static final Logger log =
    LoggerFactory.getLogger(MyClass.class);
log.info("User {} created", user.getId());
logging.level.com.example=DEBUG
```

Uses Logback by default, SLF4J facade

09. VALIDATION

Optimization

```
@NotNull @Size(min=2,max=50) String name;
@email String email;
@Min(0) int age;
public User create(@Valid @RequestBody User u...
```

Add spring-boot-starter-validation dep

```
@ControllerAdvice handles MethodArgumentNotVa...
```

12. COMMON GOTCHAS

Warning

Circular bean deps use @Lazy or constructor refactor
OpenEntityManagerInViewFilter N+1 queries
Transaction must be on public methods only
@SpringBootTest loads full context use slices
Flyway/Liquibase for DB migrations, not auto-ddl in prod